

RELATÓRIO TÉCNICO - FASE 6

SISTEMA DE COMUNICAÇÃO INTERPLANETÁRIA DA COLÔNIA (SCIC)

1. CONTEXTO DA SOLUÇÃO

A colônia Aurora Siger depende de uma rede de módulos especializados, como habitação, agricultura, comunicação, laboratório, suporte médico e armazenamento de dados, que precisam se comunicar de forma confiável apesar da distância interplanetária e das limitações de largura de banda e latência.

O Sistema de Comunicação Interplanetária da Colônia (SCIC) foi desenvolvido para organizar os dados operacionais desses módulos, medir a diferença entre o que era esperado e o que de fato acontece na rede, priorizar os alertas mais críticos, permitir consultas rápidas por prefixo e oferecer uma estimativa simples de latência futura, apoiando a tomada de decisão da equipe de operação da colônia.

O sistema foi construído utilizando exclusivamente os conteúdos estudados até esta fase: Python, Pandas, estruturas de dados (heap e trie), erro absoluto/relativo, métricas de regressão (MAE, MSE, RMSE, R^2) e conceitos de eletricidade básica e bases numéricas.

2. DESCRIÇÃO DOS DADOS

A base de dados 'dados_aurora_siger.json' foi construída de forma simulada, mas coerente com o contexto da colônia. Ela contém 100 registros, resultado de 25 módulos monitorados ao longo de 4 ciclos consecutivos cada, o que permite observar a evolução da latência de cada módulo ao longo do tempo.

2.1 ESTRUTURA DOS DADOS

Cada registro da base é composto pelos seguintes campos: id, um número inteiro que identifica unicamente o registro; nome_modulo, uma string com o nome do módulo da colônia, como "Habitacao A" ou "Comunicacao Central"; tipo_modulo, uma string que classifica o módulo em uma das seis categorias habitacao, agricultura, comunicacao, laboratorio, suporte_medico ou armazenamento_dados; e ciclo, uma string no formato ciclo_01 a ciclo_04 que indica o momento da medição, formando uma série temporal por módulo.

A latência é registrada em dois campos numéricos, ambos em milissegundos: `latencia_observada_ms`, a latência real medida na comunicação do módulo, e `latencia_prevista_ms`, a latência esperada ou estimada para aquele ciclo. As grandezas elétricas também são numéricas: `tensao_v`, a tensão do circuito de comunicação em volts; `corrente_a`, a corrente em amperes; e `potencia_w`, a potência aproximada em watts, calculada como `tensao_v * corrente_a`.

O estado operacional do módulo é descrito por `status`, que assume os valores `ativo`, `manutencao` ou `alerta`, e por `prioridade`, um número inteiro de 1 a 5 usado como chave para o heap de priorização de alertas. Por fim, `codigo_sensor` é uma string hexadecimal no formato `0xPPNN` que identifica o sensor do módulo, e `mensagem_alerta` é uma string com a descrição resumida associada ao `status` do registro.

2.2 PREFIXOS HEXADECIMAIS POR TIPO DE MÓDULO

Os códigos de sensor seguem um prefixo hexadecimal fixo de acordo com o tipo de módulo, o que facilita tanto a leitura humana quanto a busca por prefixo na trie (Seção 6): módulos de habitação usam o prefixo `0x1A`, os de agricultura usam `0x2B`, os de comunicação usam `0x3C`, os de laboratório usam `0x4D`, os de suporte médico usam `0x5E`, e os de armazenamento de dados usam `0x6F`.

2.3 ORGANIZAÇÃO, LEITURA E CADASTRO

Os dados são carregados com `json.load()` e convertidos em um `DataFrame` do `Pandas`, o que permite filtragem por tipo de módulo, por `status`, cálculo de indicadores agregados e integração direta com `scikit-learn` para o modelo de previsão. Além do carregamento da base existente, o sistema permite o cadastro manual de novos registros diretamente pelo terminal (`cadastrar_registro()`): o usuário informa nome, tipo, ciclo, latências, tensão, corrente, `status`, prioridade, código do sensor e mensagem de alerta, e o registro é adicionado ao `DataFrame` em tempo de execução, com a potência já recalculada automaticamente a partir da tensão e corrente informadas. Isso atende à exigência de que o sistema permita tanto carregar quanto cadastrar dados da colônia. As consultas e os resultados de erro são exibidos através da função `exibir_registros()`, que formata cada registro em uma linha legível (módulo, ciclo, `status`, prioridade e latências), em vez de imprimir a tabela bruta do `Pandas`, tornando as mensagens de saída mais compreensíveis para o usuário do sistema.

3. ANÁLISE NUMÉRICA E AVALIAÇÃO DE ERROS

Para cada registro foram calculados:

Erro absoluto: $|latencia_observada_ms - latencia_prevista_ms|$

Erro relativo: $erro_absoluto / latencia_prevista_ms$

O erro relativo é importante porque módulos com escalas de latência muito diferentes (por exemplo, um módulo com latência típica de 20 ms e outro de 150 ms) não podem ser comparados apenas pelo erro absoluto, um erro de 10 ms é grave no primeiro caso e pouco significativo no segundo.

Interpretação adotada no projeto:

- Erros relativos abaixo de ~10% foram tratados como dentro da margem esperada de aproximação/arredondamento e precisão numérica de sensores simulados;
- Erros entre ~15% e 30% foram tratados como indicativo de necessidade de manutenção preventiva;
- Erros acima de ~30% foram tratados como alerta crítico, indicando possível falha real na comunicação do módulo.

Essa faixa de corte foi usada para gerar os campos status e prioridade da base e para popular o heap de alertas.

4. DESCRIÇÃO DO MODELOS SIMPLES E AVALIAÇÃO DE PERFORMANCE

Foi construído um modelo simples de regressão linear (LinearRegression do scikit-learn) para estimar a latência observada de um módulo a partir do número do ciclo, aproveitando a série histórica de 4 ciclos por módulo. A ideia é simular como o sistema poderia antecipar uma tendência de degradação (ou melhora) do enlace de comunicação. Os dados foram divididos em treino e teste (train_test_split, 80/20) para evitar avaliar o modelo apenas com os dados que ele já viu.

4.1 MÉTRICAS UTILIZADAS

MAE (Erro Médio Absoluto): média das diferenças absolutas entre valor real e previsto;

MSE (Erro Quadrático Médio): penaliza mais fortemente erros grandes;

RMSE (Raiz do MSE): erro na mesma unidade da variável (ms), facilitando a interpretação;

R² (Coeficiente de Determinação): quanto da variação da latência o modelo consegue explicar.

4.2 INTERPRETAÇÃO

Um R² alto não significa, por si só, que o modelo é confiável para todos os módulos: ele indica apenas o quanto da variância dos dados de teste foi capturada pela reta ajustada. Da mesma forma, uma diferença grande entre MAE e RMSE é um sinal de alerta importante: como o RMSE penaliza mais os erros grandes, se ele estiver significativamente acima do MAE, isso indica que existem alguns registros com erro de previsão muito maior que a média, normalmente associados aos módulos que já apresentam alertas críticos na base. Por isso, o sistema não decide sozinho que um módulo está bem ou mal, pois as métricas de performance funcionam como apoio à decisão, e não como veredito final.

5. PRIORIZAÇÃO DE ALERTAS COM HEAP

Cada alerta é representado como uma tupla contendo (-prioridade, nome_modulo, mensagem_alerta). O valor de prioridade é negado porque o módulo heapq do Python implementa um min-heap, e o objetivo do sistema é que o alerta de maior prioridade seja sempre o próximo a ser tratado.

Critério de prioridade: definido a partir da magnitude do erro relativo entre latência prevista e observada, quanto maior o erro, maior a prioridade atribuída (escala de 1 a 5), refletindo o risco operacional do módulo.

Funcionamento:

1. Os alertas com status alerta ou manutencao são inseridos no heap;
2. A estrutura organiza automaticamente os elementos de forma que o de maior prioridade fique sempre acessível em tempo O(log n) de inserção/remoção;
3. A função proximo_alerta_urgente() remove e retorna o alerta mais crítico do momento, permitindo que a equipe de operação trate um alerta de cada vez, na ordem correta.

Vantagem em relação a uma lista simples: em uma lista comum, encontrar o elemento de maior prioridade exige percorrer todos os itens ($O(n)$) toda vez que um novo alerta chega.

Com o heap, tanto a inserção quanto a remoção do item mais crítico têm custo $O(\log n)$, o que se torna significativamente mais eficiente à medida que o número de alertas simultâneos cresce, e isso é algo esperado em uma colônia com muitos módulos monitorados continuamente

6. BUSCA DE REGISTROS COM TRIE

A trie foi construída para permitir busca eficiente por prefixo em dois conjuntos de dados: nomes de módulo (`nome_modulo`) e códigos de sensor (`codigo_sensor`).

Por que trie e não busca linear: em uma lista, buscar todos os nomes que começam com um prefixo (ex.: “Com”) exigiria comparar o prefixo com cada nome da base, um por um. Na trie, a busca depende apenas do tamanho do prefixo digitado, não da quantidade total de registros armazenados, ou seja, o tempo de busca não cresce com o tamanho da base, algo essencial em um sistema que pode vir a monitorar centenas de módulos.

Exemplos de uso no sistema:

Buscar “Com” retorna módulos como “Comunicacao Central”, “Comunicacao Norte” e “Comando Orbital”;

Buscar “0x3C” retorna todos os sensores de módulos do tipo comunicação, já que esse é o prefixo hexadecimal reservado para essa categoria.

7. DISPOSITIVOS, BASES NUMÉRICAS E ELETRICIDADE BÁSICA

Dispositivos de entrada: sensores simulados de tensão e corrente em cada módulo, que alimentam os campos `tensao_v` e `corrente_a` da base.

Dispositivos de saída: o próprio menu do sistema atua como dispositivo de saída, exibindo consultas, alertas priorizados e métricas. Essas informações também poderiam alimentar um monitor de status ou dashboard da colônia.

Interfaces de comunicação: os módulos poderiam se comunicar entre si e com a central via Wi-Fi ou enlaces dedicados, com o `codigo_sensor` funcionando como identificador único de cada dispositivo na rede.

Bases numéricas: cada `codigo_sensor` é representado em hexadecimal. O sistema converte esse valor para decimal com `int(codigo, 16)` e para binário com `bin()`, permitindo interpretar o identificador do sensor em diferentes representações, relevante ao lidar com protocolos de comunicação de baixo nível.

Eletricidade básica: a potência aproximada de cada módulo é calculada a partir da tensão e da corrente, usando a relação $P = V \times I$. Esse cálculo simula, por exemplo, a potência de transmissão de um módulo de comunicação a partir das leituras de seus sensores.

8. GERENCIAMENTO INTELIGENTE DA COMUNICAÇÃO

Os resultados do SCIC ajudam a entender, na prática, como funcionaria uma gestão inteligente da comunicação na Aurora Siger. Os campos de tensão, corrente e latência observada, por exemplo, representam o tipo de leitura contínua que um sistema de monitoramento real faria a cada ciclo, e é basicamente o que qualquer rede inteligente de sensores está fazendo o tempo todo, só que aqui simulado.

Essa leitura contínua só tem valor se alguém (ou algo) souber identificar quando algo está errado, e então entra o cálculo de erro relativo: quando a diferença entre a latência prevista e a observada passa de um certo limite, o módulo é automaticamente classificado como alerta. Na prática, isso é uma forma simples de detecção de anomalia, o sistema não precisa que um humano fique olhando número por número para perceber que um módulo saiu do padrão.

O heap entra logo depois disso, resolvendo o problema seguinte: uma vez que existem vários alertas, qual atender primeiro? Em vez de a equipe ter que revisar a base inteira a cada ciclo, o heap já mantém os alertas mais críticos no topo, prontos para serem tratados, isso é automação de decisão crítica, sem exigir revisão manual constante.

Além disso, há um elemento de redundância na própria estrutura da colônia: existem vários módulos de comunicação (Central, Norte, Sul, Leste, Oeste, Auxiliar) e mais de um módulo de armazenamento de dados. Isso simula justamente a ideia de que a rede não depende de um único ponto, pois se um módulo falha, os outros seguem operando.

O modelo de previsão de latência, construído a partir do histórico de ciclos, é onde a manutenção preditiva aparece de forma mais direta. Se a tendência de latência de um módulo está subindo ciclo após ciclo, dá pra agir antes que ele chegue de fato a falhar, em vez de só reagir depois do problema.

E por fim, a própria divisão da colônia em módulos de comunicação regionais (Norte, Sul, Leste, Oeste) é um paralelo direto com o conceito de microrredes, aonde cada região consegue operar de forma mais autônoma, o que deixa o sistema como um todo mais resiliente do que se tudo dependesse de uma única rede centralizada.

No fim, nenhum desses pontos exige hardware real ou inteligência artificial avançada, porque o que sustenta a discussão inteira são só os próprios dados que o SCIC já calcula: latência, erro e prioridade, bem organizados.

9. ANÁLISE FINAL CONSOLIDADA

O sistema oferece uma funcionalidade de análise final (`analise_final()`, opção 9 do menu) que reúne, em uma única execução, os principais resultados calculados pelo SCIC: o erro relativo médio de toda a base, o total de alertas e manutenções pendentes (com o alerta mais urgente identificado via heap) e o desempenho do modelo de previsão de latência (MAE, MSE, RMSE e R^2 , com a mesma interpretação textual descrita na Seção 4.3).

A partir desses três indicadores combinados, o sistema gera uma conclusão automática: se o modelo apresenta bom poder explicativo ($R^2 \geq 0,7$) e o erro relativo médio da rede está controlado (abaixo de 15%), o sistema indica que a colônia opera dentro da normalidade; caso contrário, recomenda priorizar os alertas no topo do heap e revisar os módulos mais críticos. Essa funcionalidade atende diretamente à exigência de que o sistema seja capaz de exibir uma análise final dos resultados, funcionando como uma camada de apoio à decisão acima dos indicadores individuais já discutidos nas seções anteriores.

10. REFLEXÃO SOCIAL, CULTURAL E SUSTENTÁVEL

Um dos pontos mais relevantes ao pensar na sustentabilidade do projeto é que, numa colônia interplanetária, energia e largura de banda não são recursos infinitos, pelo contrário, tendem a ser um dos maiores limitantes de qualquer operação. Nesse sentido, o SCIC contribui de forma concreta ao evitar desperdício, pois o heap garante que apenas os módulos que realmente precisam de atenção prioritária consumam esforço computacional e humano imediato, enquanto a trie evita que o sistema precise reprocessar toda a base de dados a cada consulta. Esse uso mais eficiente da infraestrutura de comunicação não é uma vantagem exclusiva de um cenário fictício na Aurora Siger, ele é o mesmo princípio que sustenta sistemas reais de comunicação na Terra, onde economizar processamento e transmissão também significa economizar energia.

Outro aspecto importante do sistema é que nenhuma das decisões que ele toma acontece de forma opaca. A priorização de um alerta, a classificação de um módulo como crítico e até a previsão de latência dependem de critérios que podem ser verificados por qualquer pessoa da equipe — erro relativo, prioridade numérica, métricas de regressão —, e não de um processo fechado que apenas devolve um resultado sem explicação. Isso importa porque decisões automatizadas só continuam sendo confiáveis enquanto for possível questioná-las: se um alerta foi priorizado errado ou um modelo indicou uma tendência estranha, alguém

precisa conseguir entender o motivo, e é justamente essa transparência que permite que a equipe humana continue no controle das decisões finais.

Por fim, vale destacar que o sistema foi pensado para ser usado por uma população potencialmente diversa, o que exigiu cuidado com a forma como ele se comunica. As mensagens de alerta e os nomes de módulos foram mantidos em uma linguagem técnica e neutra, evitando gírias, jargões excludentes ou qualquer tipo de julgamento de valor embutido no texto. Esse cuidado é o que evita que o sistema, sem querer, acabe privilegiando quem já domina um vocabulário específico ou produzindo interpretações injustas para quem não compartilha do mesmo repertório cultural ou técnico.

11. LIMITAÇÕES E POSSÍVEIS MELHORIAS

Limitações atuais:

A base de dados é simulada; não há integração com sensores físicos reais;

O modelo de previsão utiliza apenas o número do ciclo como variável preditora, o que é adequado para o nível desta fase, mas simplista frente à complexidade real de uma rede de comunicação interplanetária;

A definição dos limites de erro aceitável (10%, 30%) foi feita de forma heurística, sem calibração estatística mais robusta;

O heap e a trie operam em memória, sem persistência entre execuções.

Possíveis melhorias futuras:

Incluir mais variáveis preditoras no modelo (tipo de módulo, tensão, corrente), avaliando também com Grid Search/Random Search para ajuste de hiperparâmetros;

Persistir o estado do heap e da trie entre execuções (salvando em arquivo);

Adicionar visualizações gráficas (Matplotlib/Seaborn) mostrando a evolução da latência por módulo ao longo dos ciclos;

Expandir a reflexão social/cultural incorporando efetivamente conhecimentos tradicionais e de sustentabilidade energética na definição de prioridades do sistema.